

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA0609	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	Ranjitha. R	Reg. No.:	192421285
Section / Batch:	Information Technology	Date:	25/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

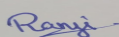
Q31.Steiner Tree Problem (NP-Hard / Constraint Optimization Problem)

Given a graph with required terminal vertices, determine the minimum-cost tree connecting them. Clearly define constraints such as connectivity and inclusion of required nodes. Analyze how constraints influence the search space and solution complexity. Develop a backtracking-based solution to explore feasible trees. Apply pruning techniques to eliminate non-optimal trees. Justify how your solution ensures minimal cost and discuss NP-Hard complexity.

Code of Conduct

I Ranjitha. R(192421285) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q31. Steiner Tree Problem — Backtracking-Based Constraint Optimization

1. Problem Definition

The Steiner Tree Problem is a graph optimization problem. Given:

- A weighted graph $G = (V, E)$
- A set of required terminal vertices $T \subseteq V$
- A cost/weight associated with every edge

The objective is to find a minimum-cost tree that connects all terminal vertices. The solution may contain additional non-terminal vertices called Steiner vertices if they help reduce the total cost.

Objective:

$$\text{Minimize } \text{Cost}(\text{Tree}) = \sum_{e \in \text{Tree}} w(e)$$

while ensuring that every required terminal is connected.

2. Constraints

The solution must satisfy the following constraints:

1. Terminal Inclusion: Every terminal vertex must be included in the tree.
2. Connectivity: There must be a path between every pair of terminal vertices.
3. Acyclicity: The selected edges must form a tree, so cycles are not allowed.
4. Steiner Vertex Constraint: Non-terminal vertices can be included when they reduce the overall cost.
5. Minimum Cost: Among all feasible trees, select the one having the smallest total edge weight.

3. Effect of Constraints on Search Space

Without constraints, a backtracking algorithm could examine a very large number of possible edge combinations.

The constraints significantly reduce the search space:

- Terminal constraint removes solutions that fail to connect required vertices.
- Cycle constraint immediately rejects edge selections that create cycles.
- Connectivity constraint eliminates disconnected partial solutions.
- Cost constraint allows expensive partial solutions to be pruned early.
- Lower-bound pruning avoids exploring branches that cannot possibly produce a better solution.

Thus, constraints make the search more manageable, although the worst-case search space remains

exponential.

4. Backtracking Algorithm

Input: Weighted graph $G(V, E)$, terminal set T

Output: Minimum-cost Steiner tree connecting all terminals.

STEINER-TREE(G, T)

bestCost $\leftarrow \infty$

bestTree $\leftarrow \emptyset$

BACKTRACK(selectedEdges, index, currentCost)

if currentCost $\geq$ bestCost

return

if containsCycle(selectedEdges)

return

if all terminals in T are connected

if selectedEdges form a tree

bestCost $\leftarrow$ currentCost

bestTree $\leftarrow$ selectedEdges

return

if index = number of edges

return

edge $\leftarrow$ G.edges[index]

// Include the current edge

add edge to selectedEdges

BACKTRACK(selectedEdges,

index + 1,

currentCost + weight(edge))

remove edge from selectedEdges

// Exclude the current edge

BACKTRACK(selectedEdges,

index + 1,

currentCost)

BACKTRACK($\emptyset$, 0, 0)

return bestTree, bestCost

5. Pruning Techniques

A. Cost-Based Pruning

If:

$$currentCost \geq bestCost$$

the branch cannot produce a better solution, so it is discarded.

B. Cycle Pruning

If adding an edge creates a cycle, that branch is immediately rejected because a Steiner solution must be a tree.

C. Connectivity Pruning

Branches that cannot possibly connect all remaining terminals are eliminated.

D. Lower-Bound Pruning

Calculate a lower bound on the additional cost required to connect the remaining terminals.

If:

$$currentCost + lowerBound \geq bestCost$$

the branch is pruned.

This is more powerful than checking only the current cost.

6. Example

Consider terminals:

$$T = \{A, C, D\}$$

Suppose the graph contains the following edges:

Edge Cost

A-B 2

B-C 2

B-D 3

A-C 7

C-D 6

A possible solution is:

$$A - B, B - C, B - D$$

Total cost:

$$2 + 2 + 3 = 7$$

This tree connects A, C, and D using B as a Steiner vertex.

The algorithm compares this with other feasible trees. Any branch whose current cost already exceeds 7 can be pruned once a cost-7 solution has been found.

Therefore, the minimum solution obtained is:

$$\boxed{Cost = 7}$$

7. Why the Solution Is Minimal

The algorithm explores feasible combinations of edges using backtracking.

For every possible branch, it considers:

- including the edge, or
- excluding the edge.

It records the lowest-cost feasible tree as `bestTree`.

Pruning does **not** remove a potentially optimal solution because a branch is discarded only when it is already invalid or when its minimum possible future cost cannot improve the current best solution.

Therefore, when the search terminates:

$$\boxed{bestCost = \text{minimum possible cost}}$$

8. Complexity and NP-Hardness

The **Steiner Tree Problem** is NP-Hard in general graphs.

If the graph contains m edges, straightforward backtracking can potentially examine approximately:

$$O(2^m)$$

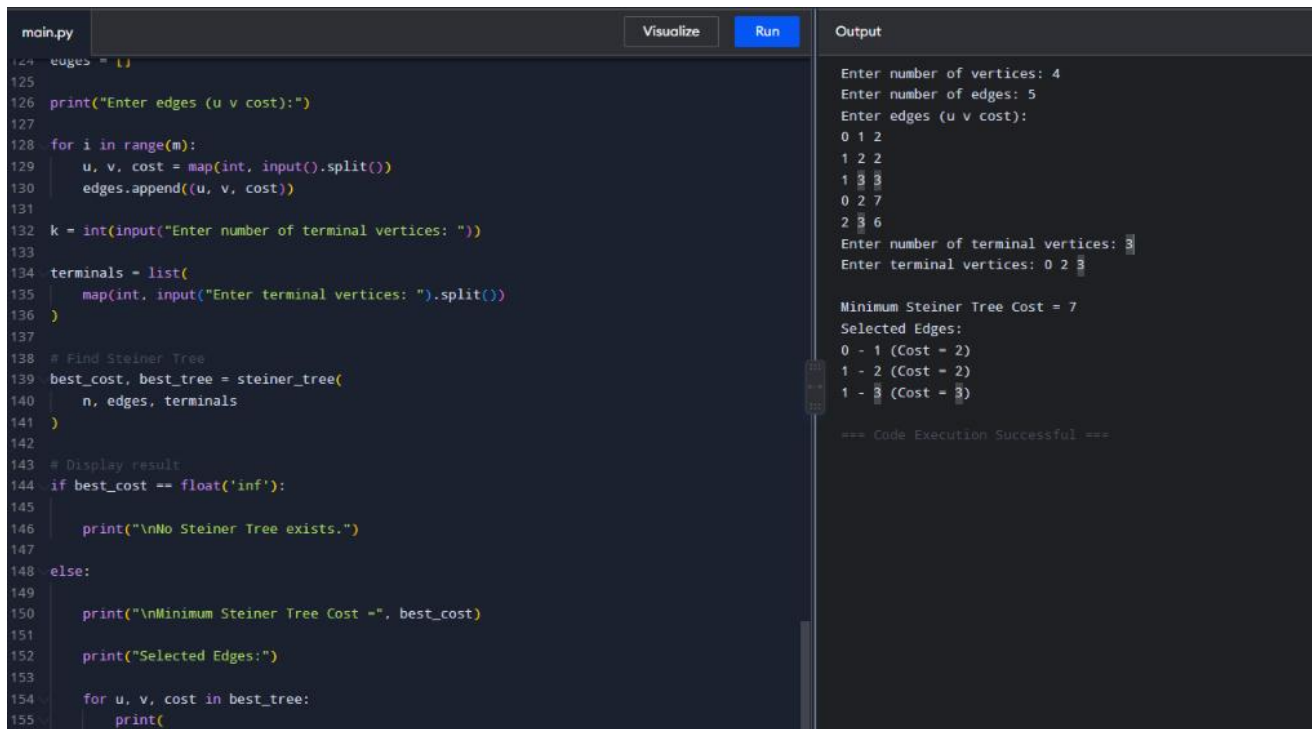
edge subsets.

For each subset, checking whether all terminals are connected can itself require graph traversal, such as DFS or BFS.

Hence, the overall worst-case complexity is exponential, making the problem impractical for large graphs using simple exhaustive backtracking.

However, pruning techniques such as cycle detection, connectivity checks, cost bounds, and lower-bound estimation can substantially reduce the practical search space.

9.output:



```
main.py Visualize Run Output
124 edges = []
125
126 print("Enter edges (u v cost):")
127
128 for i in range(m):
129     u, v, cost = map(int, input().split())
130     edges.append((u, v, cost))
131
132 k = int(input("Enter number of terminal vertices: "))
133
134 terminals = list(
135     map(int, input("Enter terminal vertices: ").split())
136 )
137
138 # Find Steiner Tree
139 best_cost, best_tree = steiner_tree(
140     n, edges, terminals
141 )
142
143 # Display result
144 if best_cost == float('inf'):
145     print("\nNo Steiner Tree exists.")
146
147 else:
148
149     print("\nMinimum Steiner Tree Cost =", best_cost)
150
151     print("Selected Edges:")
152
153     for u, v, cost in best_tree:
154         print(
155
```

```
Enter number of vertices: 4
Enter number of edges: 5
Enter edges (u v cost):
0 1 2
1 2 2
1 3 3
0 2 7
2 3 6
Enter number of terminal vertices: 3
Enter terminal vertices: 0 2 3

Minimum Steiner Tree Cost = 7
Selected Edges:
0 - 1 (Cost = 2)
1 - 2 (Cost = 2)
1 - 3 (Cost = 3)

=== Code Execution Successful ===
```

10. Conclusion

The Steiner Tree Problem demonstrates how constraint-based problem solving can be combined with backtracking and optimization. The constraints ensure that only valid trees containing all required terminals are considered. Pruning eliminates invalid or provably non-optimal branches, improving efficiency. Although these techniques can significantly reduce practical computation, the general Steiner Tree Problem remains NP-Hard, and its worst-case solution complexity is exponential.